⚠ **TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE**

This project is a **training site, not a real medical device** under any regulatory definition — it is not Software as a Medical Device (SaMD) or Software in a Medical Device (SiMD). This page is not a genuine IEC 62304 deliverable: it illustrates the *kind* of document a real SaMD/SiMD project would produce. See the document register for the full explanation.

Status: Partial — see Gaps. Clause: 5.5 — Unit Implementation and Verification · **Register:** Document Register

What the standard requires (Class C — every sub-clause of 5.5)

REF	TITLE	APPLIES AT	OUTPUT
5.5.1	Implement each software unit	A, B, C	Implemented software units
5.5.2	Establish unit verification process	**B, C**	Strategies, methods and procedures for verifying units, with test procedures evaluated for adequacy where verification is by testing
5.5.3	Unit acceptance criteria	**B, C**	Acceptance criteria for units, established before integration, and evidence the units meet them
5.5.4	Additional unit acceptance criteria	**C only**	Where present in the design: event sequence, data/control flow, resource allocation, fault handling, variable initialisation, self-diagnostics, memory management, boundary conditions
5.5.5	Software unit verification	**B, C**	Documented results of software unit verification

5.5.1 — Implementation

Every unit named in 04 — Architecture is implemented and shipped: `nav.js`, `async-utils.js`, `theme.js`, `learn.js`, `quiz.js`, `contact.js`, `style.css`. Not in question — the gap in this document is 5.5.2–5.5.5, verification, not 5.5.1.

The actual gap, stated plainly

`tests/test_site.py` says this about itself, in its own docstring: *"It is a black-box test suite. It does not import or inspect the site's JavaScript; it drives the pages the way a person would — clicking, typing, reading what appears — and asserts on the result."*

That is a deliberate and good design for **system testing** (see 08) — but it means this project has **no unit-level test suite**. There is no Jest, no Vitest, no isolated call into `deliverablesFor()` or `classify()` with a hand-built input and an asserted output; every one of those functions is verified only through its visible effect on a rendered page, several layers away from the function itself.

5.5.2–5.5.5, as they actually stand

- **5.5.2 (verification process):** the process that exists is *review*, not test — a person (or an AI coding agent under review) reads the function and checks it against the requirement. That is one of the standard's own acceptable methods (`code review, static analysis, unit testing, or a combination`, per `data/phases.json`'s advanced detail for this clause), but it is the weakest of the three on its own, and it is what this project actually has.
- **5.5.3 (acceptance criteria before integration):** criteria exist informally — a function is expected to do what its name and comment say — but are not written down as a separate, checkable list before the function is used elsewhere.
- **5.5.4 (Class C's additional criteria — boundary conditions, fault handling, etc.):** partially covered *incidentally* by system tests that happen to exercise a boundary (empty JSON, a 404, a malformed response — see 08), but not because a unit-level boundary-condition checklist drove them there.

- **5.5.5 (documented results):** what exists is system-test results, not unit-test results — real evidence, but evidence of a different, coarser claim ("the page behaves correctly") than 5.5.5 asks for ("this unit behaves correctly").

Why this gap is left open rather than closed with a token unit suite

A unit test suite added *after* the fact, purely to make this table look complete, would verify functions no differently than the system tests already do — most of `learn.js`'s functions have no meaning independent of the DOM they render into, so a "unit" test would end up re-implementing a slice of what Playwright already drives, at a maintenance cost with limited new information. The honest record of that trade-off is more useful to a learner than a unit suite that exists to fill a cell in a table.

Gap

No unit-level verification exists, distinct from the system-level verification in 08. For a real Class C project this is a real, reportable finding, not a stylistic choice — the standard asks for both because a system test passing does not prove every individual unit was tested at its own boundaries, only that the system as a whole produced the right answer for the specific paths the system test happened to take.